OSINT & DLP

SYSTEM

Grupo Roullier · Documentação Técnica Completa

Versão 2.1 | Confidencial · Uso Interno

Grupo Roullier Security Team · 2026

Sumário

01 O Que é o OSINT & DLP System
02 Glossário de Termos
03 Arquitetura e Componentes
04 Fluxo de Operação Detalhado
05 Intelligence Matrix
06 Detecção, Classificação e Categorização
07 Modos de Operação
08 Instalação Passo a Passo
09 Configuração de Variáveis de Ambiente
10 Execução e Comandos
11 Dashboard Web · Guia Visual
12 Alertas Telegram
13 API REST · Referência Completa
14 Segurança e Privacidade
15 Manutenção e Ingestão de Dados

01. O Que é o OSINT & DLP System

O nome do sistema é composto por dois acrônimos amplamente reconhecidos na área de segurança da informação:

- OSINT (Open Source Intelligence) · Inteligência em Fontes Abertas: a prática de coletar e analisar informações disponíveis publicamente na internet.
- DLP (Data Leak Prevention) · Prevenção de Vazamento de Dados: conjunto de técnicas para detectar e impedir a exposição não autorizada de informações sensíveis.

Combinados, o OSINT & DLP System é uma plataforma automatizada que varre continuamente a internet pública em busca de documentos, planilhas e arquivos que possam conter informações confidenciais do Grupo Roullier. O sistema monitora as três empresas principais do grupo no Brasil:

- Timac Agro Brasil · Indústria e Comércio de Fertilizantes
- Sulfabrás Sulfatos do Brasil · Fabricação de Sulfatos e Químicos
- Phosphea Brasil · Comércio de Fosfatos

Quando o sistema encontra um documento suspeito, ele o baixa diretamente na memória do servidor (sem salvar em disco), extrai o texto, analisa com expressões regulares, classifica o nível de risco e notifica a equipe de segurança. Tudo isso de forma totalmente automática, rodando 24/7.

02. Glossário de Termos

03. Arquitetura e Componentes

O sistema é construído em Python 3.10+ e segue uma arquitetura modular em camadas independentes. Dois modos de operação (Server e Desktop) compartilham o mesmo núcleo de lógica garantindo resultados idênticos.

3.1 Mapa de Componentes

3.2 Estrutura de Pastas

```
Doc-Tracker/
+-- config/                  Configuracoes compartilhadas
|   +-- settings.py          Variaveis de ambiente
```

```
|   +-- intelligence_matrix.py  Matriz de inteligencia
+-- core/                     Banco de dados
|   +-- database.py           Engine async SQLite
|   +-- models.py             Modelos Scan e Finding
+-- crawler/                  Motor de busca
|   +-- dork_generator.py     80+ consultas automaticas
|   +-- search_engine.py      Busca via DuckDuckGo
|   +-- url_filter.py         Filtro de dominios e deteccao
+-- inspector/                Inspecao de documentos
|   +-- downloader.py         Download em memoria
|   +-- extractor.py          Extracao PDF/DOCX/XLSX/TXT
|   +-- regex_engine.py       Deteccao de PII por regex
|   +-- risk_classifier.py    Classificacao de risco
+-- alerts/                   Webhooks
|   +-- webhook.py            Telegram alerts
+-- api/                      Dashboard e API
|   +-- app.py                FastAPI application
|   +-- routes/               Endpoints REST
|   +-- templates/            HTML do dashboard
+-- static/                   Frontend
|   +-- css/dashboard.css     Estilos dark mode
|   +-- js/dashboard.js       Logica do dashboard
+-- server/                   Modo servidor
|   +-- run_server.py         Entry point + scheduler
|   +-- osint-dlp.service     Servico systemd
+-- desktop/                  Modo desktop
|   +-- app.py                GUI Tkinter
|   +-- run_desktop.py        Entry point
+-- data/entities.json        Dados das entidades
+-- documentacao/             Esta documentacao
+-- requirements.txt          Dependencias Python
+-- .env.example              Template de configuracao
+-- run.py                    Entry point dev local
```

04. Fluxo de Operação Detalhado

Cada scan completo segue esta sequência de 7 etapas:

ETAPA 1 · Geração de Dorks

O DorkGenerator cria aproximadamente 80 consultas de busca avançadas. Cada dork combina nomes de empresas, CNPJs, CPFs, fornecedores e termos sensíveis com operadores de busca para plataformas específicas (Scribd, SlideShare, Issuu, GitHub, GitLab, Google Drive). Domínios oficiais do grupo são automaticamente excluídos com o operador -site:.

ETAPA 2 · Varredura Web

O SearchEngine executa cada dork via DuckDuckGo com delays randomizados de 2 a 4 segundos entre consultas, evitando bloqueio por rate-limiting. Cada resultado retorna título, URL e snippet.

ETAPA 3 · Deduplicação

Antes de processar cada URL, o sistema verifica no banco de dados se ela já foi analisada anteriormente. URLs já conhecidas são ignoradas, garantindo que apenas novos documentos sejam processados.

ETAPA 4 · Download em Memória

O Downloader obtém cada documento diretamente em memória (BytesIO) via httpx, sem gravar nenhum arquivo no disco. Isso minimiza a pegada forense e acelera o processamento.

ETAPA 5 · Extração de Texto e Metadados

O TextExtractor identifica o tipo de arquivo e processa: PDF via PyMuPDF, DOCX via python-docx, XLSX via openpyxl, e texto puro. Além do conteúdo, extrai metadados como autor, software criador e data de criação, filtrando artefatos técnicos (nomes de ferramentas como Microsoft Office).

ETAPA 6 · Inspeção por Regex e Classificação

O RegexEngine analisa o texto em busca de CPFs, CNPJs, dados financeiros, entidades do grupo e termos sensíveis. Os snippets são mascarados para proteger PII. O RiskClassifier então atribui um score numérico (1-100) e nível (Crítico/Alto/Médio/Baixo).

ETAPA 7 · Persistência e Alertas

Os findings são salvos no banco SQLite. O país de origem é detectado pelo TLD do domínio e o idioma por análise textual. Ao final do scan, um resumo com todos os novos links é enviado via Telegram, agrupado por nível de risco.

05. Intelligence Matrix

A Intelligence Matrix é a base de conhecimento central do sistema. Ela é carregada dinamicamente do arquivo data/entities.json, que contém dados extraídos das fichas cadastrais do grupo.
Dados monitorados na versão atual:

Para atualizar a matrix com novas entidades, basta adicionar novos PDFs cadastrais na pasta anexos/ e executar o script de ingestão (Seção 15).

06. Detecção, Classificação e Categorização

6.1 Níveis de Risco

6.2 Categorias de Finding

6.3 Filtros Regionais

O sistema detecta automaticamente o país de origem pela extensão do domínio (TLD) e o idioma do conteúdo por análise textual. Isso permite filtrar findings por região no dashboard.

07. Modos de Operação

7.1 Modo Server (Produção)

O modo Server é a versão principal, projetada para ambientes de produção com monitoramento 24/7.
Recursos exclusivos:

- Dashboard web com tema cybersecurity dark mode (glassmorphism, neon, gradientes)
- Gráficos interativos: distribuição por risco (doughnut) e fontes de vazamento (barras)
- Tabela de triagem com filtros por risco, status, categoria, país e idioma
- Barra de progresso em tempo real com percentual durante scans
- Agendamento automático via APScheduler (padrão 08h, 14h, 22h BRT)
- Alertas Telegram: startup do sistema, início de scan e resumo com links novos
- SSE (Server-Sent Events) para push notifications no dashboard
- API REST completa para integrações externas
- Serviço systemd para operação contínua sem login (Linux)

7.2 Modo Desktop (Consulta Ad-hoc)

O modo Desktop é uma aplicação GUI standalone construída com Tkinter, para consultas pontuais sem necessidade de servidor web.

- Interface gráfica dark mode com barra de progresso e métricas
- Exportação de resultados para CSV e JSON
- Operação independente — não precisa de servidor web rodando
- Compilável para .exe standalone via cx_Freeze

08. Instalação Passo a Passo

8.1 Requisitos do Sistema

8.2 Passo 1: Clonar o Repositório

```
git clone <url-do-repositorio>
cd Doc-Tracker
```

8.3 Passo 2: Criar Ambiente Virtual

Linux / macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Windows:

```
python -m venv venv
venv\Scripts\activate
```

8.4 Passo 3: Instalar Dependências

```
pip install -r requirements.txt
```

8.5 Passo 4: Configurar Variáveis de Ambiente

```
cp .env.example .env
nano .env   # editar com valores reais
```

Veja a Seção 09 para detalhes de cada variável.

8.6 Passo 5: Executar

Escolha o modo de execução adequado (Seção 10).

8.7 Passo 6 (Opcional): Instalar como Serviço

Para servidores Linux em produção, instale como serviço systemd:

```
# Editar o arquivo de servico para ajustar caminhos e usuario:
nano server/osint-dlp.service

# Copiar e ativar:
sudo cp server/osint-dlp.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable osint-dlp
sudo systemctl start osint-dlp
```

Verificar status:

```
sudo systemctl status osint-dlp
sudo journalctl -u osint-dlp -f
```

09. Configuração de Variáveis de Ambiente

O arquivo .env contém todas as configurações sensíveis. Ele NUNCA deve ser enviado ao repositório Git (já está no .gitignore). Use o .env.example como template.

10. Execução e Comandos

11. Dashboard Web · Guia Visual

O dashboard segue padrões de design de ferramentas de cybersecurity profissionais: tema dark mode, glassmorphism, cores neon para status, tipografia Inter e JetBrains Mono.

11.1 Seções do Dashboard

12. Alertas Telegram

O sistema envia 3 tipos de alertas via Telegram:

12.1 Tipos de Alerta

12.2 Comportamento Incremental

O sistema utiliza deduplicação por URL: se um documento já foi encontrado em um scan anterior, ele não é reprocessado nem notificado novamente. Isso garante que apenas NOVOS vazamentos geram alertas nos scans subsequentes.

13. API REST · Referência Completa

14. Segurança e Privacidade

15. Manutenção e Ingestão de Dados

Para adicionar novas empresas ou atualizar dados cadastrais:

1. Coloque os PDFs das fichas cadastrais na pasta anexos/ (na raiz do projeto).
2. Execute o script de ingestão:

```
python tools/ingest_anexos.py
```

3. O script extrai automaticamente:
   - Razão social e CNPJ principal
   - Inscrição estadual, data de fundação, capital social
   - Sócios com percentual de participação
   - Administradores com CPFs
   - Filiais com CNPJs e endereços
   - Todos os CNPJs e CPFs mencionados no documento
   - Emails, telefones, fornecedores e referências bancárias
4. Os dados são salvos em data/entities.json e os PDFs originais são deletados.
5. O próximo scan usará automaticamente as novas entidades.

Termo	Significado
Dork	Consulta de busca avançada (Google Dork) que utiliza operadores especiais para encontrar resultados específicos
Finding	Um resultado de varredura · um documento ou página web que contém dados sensíveis detectados
Scan	Uma execução completa de varredura: geração de dorks · busca · download · inspeção · classificação

Score | Pontuação numérica (1-100) atribuída a cada finding indicando a gravidade do risco
Snippet | Trecho mascarado do texto onde dados sensíveis foram encontrados
Triagem | Processo de analisar e classificar cada finding como legítimo, falso positivo ou resolvido
Intelligence Matrix | Base de conhecimento com todas as entidades, CNPJs, CPFs e termos monitorados
Crawler | Motor de busca automática que executa as consultas e coleta URLs
Inspector | Módulo que baixa, extrai texto, inspeciona e classifica documentos
Soft Delete | Método de exclusão que marca registros como inativos sem removê-los fisicamente
SSE | Server-Sent Events · tecnologia para enviar notificações em tempo real do servidor para o navegador
TLD | Top-Level Domain · sufixo do domínio (.com.br, .pt, .fr) usado para detectar país de origem

Módulo	Pasta	Função	Tecnologia
Configuração	config/	Settings globais e Intelligence Matrix	Pydantic Settings
Core	core/	Modelos de dados e engine de banco	SQLAlchemy Async + aiosqlite
Crawler	crawler/	Geração de dorks e motor de busca	DuckDuckGo Search (ddgs)
Inspector	inspector/	Download, extração de texto, regex, risco	httpx, PyMuPDF, python-docx, openpyxl
Alertas	alerts/	Notificações Telegram	httpx
API	api/	Dashboard web e API REST	FastAPI + Jinja2 + Chart.js
Server	server/	Entry point do modo servidor	Uvicorn + APScheduler
Desktop	desktop/	Interface gráfica standalone	Tkinter
Tools	tools/	Scripts utilitários (ingestão, documentação)	Python stdlib
Data	data/	Entidades extraídas dos cadastros	JSON

Categoria	Quantidade	Exemplos
Entidades (Razões Sociais)	19 nomes	TIMAC AGRO INDÚTRIA E COMÉRCIO DE FERTILIZANTES LTDA
CNPJs	24 números	02.329.713/0001-29, 26.769.908/0001-58
CPFs	Administradores	Sociós e diretores do grupo
Fornecedores	21 empresas	OCP Brasil, Pisani Plásticos, MOVIDA
Pessoas-chave	4 nomes	Administradores e sócios registrados
Emails Corporativos	9 endereços	nfe1@timacagro.com.br
Termos Sensíveis	15+ termos	confidencial, uso interno, senha, password

Nível	Emoji	Score	Critérios de Classificação
Crítico	vermelho	80-100	Múltiplos CPFs + CNPJs + dados financeiros, ou CPFs expostos com contexto altamente sensível
Alto	laranja	60-79	CPFs ou CNPJs do grupo combinados com termos financeiros
Médio	amarelo	40-59	Menção a entidades do grupo com termos sensíveis moderados
Baixo	verde	1-39	Menção genérica a entidades ou termos isolados sem contexto crítico

Categoria	Descrição	Exemplo de Detecção
RH	Dados de funcionários e trabalhistas	Folha de pagamento com CPFs de colaboradores
Financeiro	Dados monetários e contábeis	Balanço patrimonial, faturamento, contratos
TI	Infraestrutura tecnológica	Configurações de rede, diagramas de sistema
TI/Segurança	Credenciais e acessos	Senhas, tokens de API, chaves VPN
Dados Pessoais	PII sem contexto corporativo	CPFs e endereços em listagens públicas
Corporativo	Documentos estratégicos	Propostas comerciais, contratos de fornecimento

País	TLDs Detectados	Idiomas
Brasil	.com.br, .org.br, .gov.br	Português (pt)
Portugal	.pt	Português (pt)
França	.fr	Francês (fr)
EUA	.us, .gov	Inglês (en)
Espanha	.es	Espanhol (es)
Internacional	Todos os demais	Detectado por análise

Requisito	Mínimo	Recomendado
Python	3.10	3.12+
Sistema Operacional	Linux ou Windows	Ubuntu 22.04 LTS (Server)
RAM	512 MB	1 GB

Disco | 100 MB | 500 MB (banco pode crescer)
Rede | Acesso à internet | Sem restrições de firewall para HTTPS

Variável	Descrição	Padrão	Obrigatória
DATABASE_URL	URL de conexão com o banco SQLite	sqlite+aiosqlite:///osint_dlp.db	Sim
API_HOST	Host em que o servidor escuta	0.0.0.0	Não
API_PORT	Porta do servidor web	8443	Não
TELEGRAM_BOT_TOKEN	Token do bot Telegram (obtido via @BotFather)	·	Para alertas
TELEGRAM_CHAT_ID	ID do chat/grupo Telegram para receber alertas	·	Para alertas
SCAN_SCHEDULE_HOURS	Horários de scan automático (separados por vírgula)	8,14,22	Não
SERVER_MODE	Ativar funcionalidades de servidor (scheduler, alerts)	true	Não
DASHBOARD_URL	URL pública do dashboard (usada nos links do Telegram)	http://localhost:8443	Não
PROXY_URL	Proxy SOCKS5 para crawling (ex: socks5://host:port)	·	Não

Modo	Comando	Descrição
Desenvolvimento Local	python run.py	Servidor web local sem scheduler nem Telegram
Servidor (Produção)	python server/run_server.py	Dashboard + Telegram + Auto-scan 3x/dia
Desktop (GUI)	python desktop/run_desktop.py	Interface gráfica standalone
Compilar Desktop	python setup_desktop.py build	Gera executável .exe via cx_Freeze
Ingestão de Dados	python tools/ingest_anexos.py	Processa novos PDFs cadastrais
Gerar Documentação	python tools/generate_docs.py	Recria estes documentos

Seção	Descrição
Header	Logo, status do sistema (Idle/Scanning com dot pulsante), botão 'Iniciar Scan'
Barra de Progresso	Aparece durante scans com fase atual, percentual e detalhe do dork/URL
Métricas	5 cards com Total Findings, Crítico, Alto, Médio e Baixo (hover com animação)
Gráfico de Risco	Doughnut chart mostrando distribuição por nível de risco
Fontes de Vazamento	Bar chart horizontal com as plataformas que mais contêm vazamentos
Tabela de Triagem	Lista completa de findings com 13 colunas filtráveis
Painel de Detalhes	Slide-in lateral com informações completas do finding selecionado
Ações	Botões para classificar: Investigar, Falso Positivo, Resolvido, Notificado
Paginação	Navegação entre páginas de resultados (20 por página)
Toasts	Notificações flutuantes para confirmações de ações e novos alertas

Alerta	Quando	Conteúdo
Sistema Online	Ao iniciar o serviço	URL do dashboard, horário, schedule de scans configurado
Scan Iniciado	Início de cada scan	Número de dorks a executar, timestamp
Resumo do Scan	Final de cada scan	Contagem por nível + lista de TODOS os novos links agrupados por risco

Método	Endpoint	Descrição
GET	/	Dashboard HTML completo
GET	/api/dashboard	Métricas agregadas: totais, por risco, por plataforma, por categoria, último scan
GET	/api/findings?page=N&risk_level=X&status=Y&category=Z&country=C&language=L	Listar findings com paginação e filtros
PATCH	/api/findings/{id}/status	Atualizar status: pending, investigating, false_positive, resolved, notified
DELETE	/api/findings/{id}	Soft delete (marca is_deleted=True, mantém no banco)
GET	/api/scans	Histórico dos últimos 50 scans
POST	/api/scans/trigger	Disparar scan manual (rejeita se já houver um rodando)
GET	/api/scans/progress	Progresso atual: fase, current, total, detalhe
GET	/api/stream	Stream SSE com eventos new_finding em tempo real

Medida	Descrição
Download em Memória	Documentos são baixados em BytesIO e processados sem tocar o filesystem
Mascaramento de PII	CPFs, CNPJs e emails são mascarados nos snippets (ex: ***.456.789-**)
Domínios Excluídos	Sites oficiais (timacagro.com.br, phosphea.com, roullier.com) nunca são varridos
Soft Delete	Dados nunca são removidos fisicamente · marcados como inativos para auditoria
.env Protegido	Credenciais sensíveis ficam fora do repositório via .gitignore
Delays Anti-bloqueio	Intervalos randomizados (2-4s) entre requisições de busca

Dedup por URL | Cada URL é processada apenas uma vez em todo o histórico